

Programming Methodology

Lab 7: Observer Pattern

Wednesday, October 21, 2026

New Groups!

- Important skill to work effectively with new people
- Get to know some more of your classmates :)
- **Your lab leaders will now separate you into new groups**

Reminders

- Homework 6 is due tonight (10/21) at 11:59 pm.
  - Come to office hours for help!
- If you need to miss lab for a valid and documented reason:
  - Fill out the excusal form on Canvas before the lab.
  - Waking up late or the bus being late are not excusable reasons.
- Submit what you have at the end of lab to Gradescope.
  - Lab submissions are due at 6:00 pm.
  - The submission counts towards your lab grade.

Observer Pattern Review

The Observer design pattern has two main components: **Subjects** and **Observers**.

- **Subjects are like publishers**.
  - They broadcast information to their “audience”.
  - The members of this audience are Observers.
- **Observers subscribe to Subjects**.
  - Subscribing to a Subject allows an Observer to receive its broadcasts.

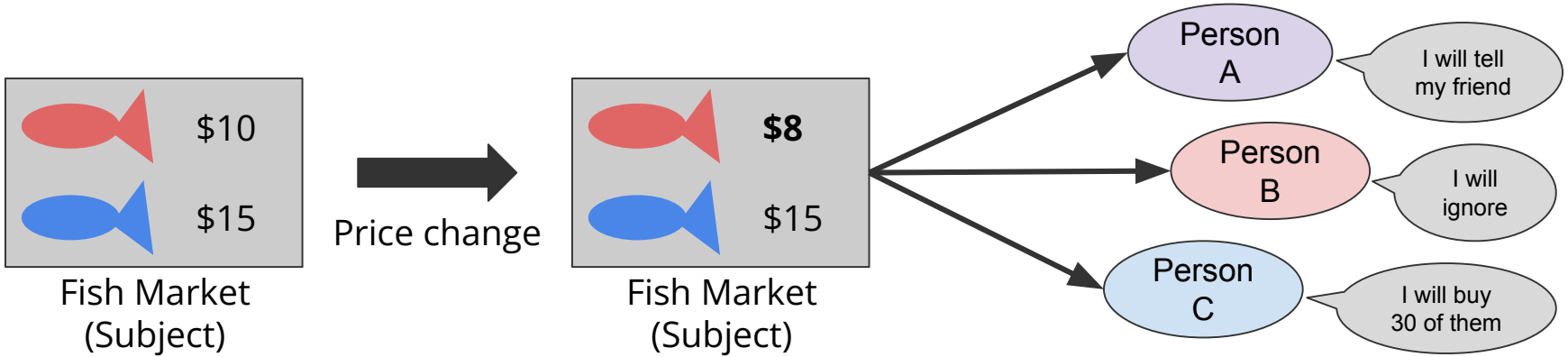
This pattern **decouples** the **data source** (Subject) from its **consumers** (Observers).

- Subject only interacts with a specific interface for notifying Observer of updates

Observer Pattern Review

For example, we could have a Subject for the price of various fish in a market.

- Each time the price of a fish changes, we broadcast the new price to anyone who is interested.
- Specifically, people (Observers) can indicate their interest by subscribing to our fish market (Subject).
    - They can then use the fish price information in different ways.



Observer Pattern Review

```
interface Subject {
  // add the given subscriber to the set of subscribers
  subscribe(subscriber: Observer): void;

  // remove the given subscriber from the set of subscribers
  unsubscribe(subscriber: Observer): void;
}
```

The Observer design pattern is implemented differently across contexts.

- In this class we will assume Subjects have the above interface.
- Observers do not have a set interface and will be defined based on context.
    - We will typically use objects to represent both Subjects and Observers, but there are cases where it makes more sense for Observers to be functions.

Exercise: Observer Pattern

Headline notifications from news outlets follow the observer pattern.

In this exercise, you are given the following interfaces:

```
interface Reader {
  // alert reader with news headline
  alert(headline: string): void;
}

interface NewsOutlet {
  // add a Reader to the set of subscribers
  subscribe(reader: Reader): void;

  // remove a Reader from the set of subscribers
  unsubscribe(reader: Reader): void;
}
```

Exercise: Observer Pattern

Write a class **`NewspaperApp`** that implements **`NewsOutlet`**. Every subscriber should receive breaking news updates, and only subscribers who have opted in should receive technology news updates. Specifically, you will need to complete the following:

- Initialize state
- **`subscribe`**: Subscribe a new **`Reader`** (consider **`techOptIn`** values)
- **`unsubscribe`**: Unsubscribe a **`Reader`** from all updates
- **`unsubscribeFromTechNews`**: Unsubscribe a **`Reader`** from tech news
- **`_notifyBreakingNews`**: Notify all users of a breaking news headline
- **`_notifyTechNews`**: Notify opted-in users of a tech news headline

You will need to ensure that a **`Reader`** instance can only be subscribed to a feed from **`NewsOutlet`** once at any given time, preventing duplicate alerts.

Solution: Observer Pattern

```
class NewspaperApp implements NewsOutlet {
  private _subscribers: Set<Reader> = new Set<Reader>();
  private _techSubscribers: Set<Reader> = new Set<Reader>();
  ...
}
```

Note that a constructor is not required when the class has no parameters.

- You can initialize the attributes directly.

Solution: Observer Pattern

```
class NewspaperApp implements NewsOutlet {
  private _subscribers: Set<Reader> = new Set<Reader>();
  private _techSubscribers: Set<Reader> = new Set<Reader>();

  // subscribe a Reader to the appropriate headline feed(s)
  subscribe(reader: Reader, techOptIn: boolean = false): void {
    this._subscribers.add(reader);
    if (techOptIn) {
      this._techSubscribers.add(reader);
    }
  }
  ...
}
```

Solution: Observer Pattern

```
class NewspaperApp implements NewsOutlet {
  private _subscribers: Set<Reader> = new Set<Reader>();
  private _techSubscribers: Set<Reader> = new Set<Reader>();
  ...
  // unsubscribe a Reader from all headline feeds
  unsubscribe(reader: Reader): void {
    this._subscribers.delete(reader);
    this._techSubscribers.delete(reader);
  }
  ...
}
```

Solution: Observer Pattern

```
class NewspaperApp implements NewsOutlet {
  private _subscribers: Set<Reader> = new Set<Reader>();
  private _techSubscribers: Set<Reader> = new Set<Reader>();
  ...
  // unsubscribe a Reader from the technology news headline feed
  unsubscribeFromTechNews(reader: Reader): void {
    this._techSubscribers.delete(reader);
  }
  ...
}
```

Solution: Observer Pattern

```
class NewspaperApp implements NewsOutlet {
  private _subscribers: Set<Reader> = new Set<Reader>();
  private _techSubscribers: Set<Reader> = new Set<Reader>();
  ...
  // notify all subscribers of a breaking news headline
  _notifyBreakingNews(headline: string): void {
    for (const reader of this._subscribers) {
      reader.alert(headline);
    }
  }
  ...
}
```

Recall: `interface Reader { alert(headline: string): void; }`

Solution: Observer Pattern

```
class NewspaperApp implements NewsOutlet {
  private _subscribers: Set<Reader> = new Set<Reader>();
  private _techSubscribers: Set<Reader> = new Set<Reader>();
  ...
  // notify all opted-in subscribers of a technology news headline
  _notifyTechNews(headline: string): void {
    for (const reader of this._techSubscribers) {
      reader.alert(headline);
    }
  }
}
```

Solution: Observer Pattern

```
class NewspaperApp implements NewsOutlet {
  private _subscribers: Set<Reader>
    = new Set<Reader>();
  private _techSubscribers: Set<Reader>
    = new Set<Reader>();

  subscribe(
    reader: Reader,
    techOptIn: boolean = false
  ): void {
    this._subscribers.add(reader);
    if (techOptIn) {
      this._techSubscribers.add(reader);
    }
  }

  unsubscribe(reader: Reader): void {
    this._subscribers.delete(reader);
    this._techSubscribers.delete(reader);
  }

  unsubscribeFromTechNews(
    reader: Reader
  ): void {
    this._techSubscribers.delete(reader);
  }

  private _notifyBreakingNews(
    headline: string
  ): void {
    for (const reader of this._subscribers) {
      reader.alert(headline);
    }
  }

  private _notifyTechNews(headline: string): void {
    for (const reader of this._techSubscribers) {
      reader.alert(headline);
    }
  }

  ...
}
```